

Неделя 3: Монолит vs модульность

Архитектура ПО / Software Architecture

Цели сегодняшней лекции

К концу лекции вы сможете:

1. Объяснить, что такое **монолит** (*monolith*) и когда он — нормальный выбор
2. Отличить **модульный монолит** (*modular monolith*) от «всего в одной куче»
3. Узнать антипаттерн «**большой ком грязи**» (*big ball of mud*) (без стыда — с планом)
4. Оценить **влияние изменений**: трогаем одно место или много (**сцепление** (*coupling*), **связность** (*cohesion*) — простыми словами)

План (80 минут)

1. Мост: качества → структура
2. Монолит: одна единица развёртывания
3. Модульный монолит: границы внутри
4. Big ball of mud
5. Влияние изменений; coupling / cohesion
6. Когда оставить простой монолит

Практика сегодня — **раздатка Markdown** (не слайды): режем свалку фич на модули.

Зачем говорить о модулях сейчас

На неделе 2 вы выбирали топ-3 качества (*quality attributes*). Структура отвечает на вопрос:

- Насколько легко **менять** одно, не ломая другое? (*maintainability*)
- Можно ли развивать части **параллельно**?
- Где «болит», если всё свалено вместе?

Не путать с микросервисами

Сегодня не про «обязательно разрезать на 20 сервисов».

- Сначала: один процесс / одно приложение — но с ясными **модулями**
- Микросервисы — отдельная (и дорогая) история; позже, если понадобится

Частая ошибка

Считать, что «архитектура = сразу микросервисы». Часто достаточно аккуратного монолита.

Монолит (*monolith*) — рабочее определение

Монолит

Система, которую обычно **собирают и разворачивают как одно целое**: один артефакт (или тесно связанные), один основной процесс приложения.

- Пользователь может видеть сайт + приложение — но «бэкенд» часто один
- Проще отлаживать «всё сразу» на старте
- Менять «только кусочек в проде» сложнее, чем при отдельных сервисах

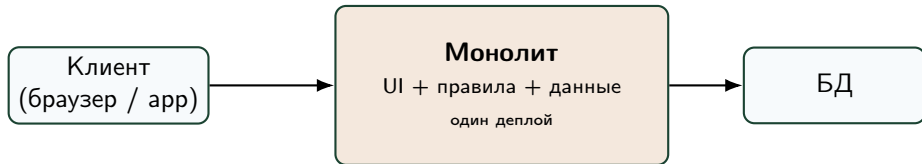
Когда монолит — нормально

- Маленькая команда, один продукт, ранний этап
- Требования ещё плывут — границы модулей не устоялись
- Важны скорость поставки и простота операций (один деплой)

Запомните

Монолит $\neq$ «плохой код». Плохо — когда внутри нет границ и всё зависит ото всего.

Картинка: один деплой



Викторина

Что точнее всего описывает монолит на этом курсе?

1. Обязательно уродливый код без структуры
2. Приложение, которое обычно разворачивают как одно целое
3. Всегда набор из десятков независимых сервисов
4. Только мобильное приложение без сервера

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Ключ — **одна (главная) единица развёртывания**, а не «плохость» кода.

Внутри монолит может быть аккуратным или хаотичным — это уже про модульность.

Модуль (*module*) — часть системы с:

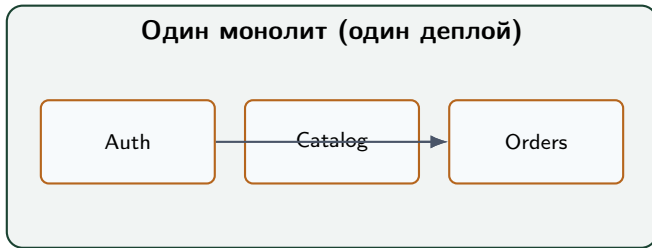
- понятным **именем** и зоной ответственности
- относительно ясной **границей** (что снаружи можно вызывать)
- своими данными/правилами насколько возможно «рядом»

Примеры имён: Auth, Catalog, Orders, Payments, Notifications.

Модульный монолит (*modular monolith*)

- Всё ещё **один** деплой / одно приложение
- Внутри — модули с границами (пакеты, папки, проекты в одном solution)
- Цель: менять заказы, не переписывая вход; тестировать куски отдельно

Это часто «золотая середина» для учебных и многих реальных продуктов.



Стрелки — разрешённые вызовы между модулями; не «каждый файл импортирует каждый».

Правила границы (интуиция)

- Модуль не лезет во внутренности другого (минуя его «вход»)
- Общие утилиты — ок; «бог-класс на всё» — нет
- Если два модуля постоянно правят одни и те же таблицы «в обход» — граница слабая

Попробуйте

На практике: для каждого модуля напишите одной фразой «отвечает за ...» и «не отвечает за ...».

Викторина

Модульный монолит — это:

1. Обязательно много отдельных серверов в проде
2. Одно развёртываемое приложение с ясными внутренними модулями
3. Запрет на любые папки в проекте
4. То же самое, что big ball of mud

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Один деплой + границы внутри.

Микросервисы — когда модули ещё и *отдельно* разворачивают (не тема обязательная сегодня).

«Большой ком грязи» (*big ball of mud*)

Антипаттерн: система, где

- нет устойчивых границ модулей
- всё зависит ото всего
- любое изменение — лотерея

Часто вырастает из «сначала быстрее написать», дедлайнов, смены людей.

- Почти у каждой команды бывал кусок «грязи»
- Важно: **заметить** и решить, копать ли границы сейчас
- Первый шаг часто — нарисовать модули на бумаге, а не сразу микросервисы

Частая ошибка

Стыдить авторов старого кода. Лучше: «где болит при изменениях?» и план упрощения.

- Чтобы поменять текст письма, правите оплату и вход
- Новый человек неделю ищет, «куда вставить фичу»
- Тесты почти невозможны без поднятия «всего мира»
- Имена файлов вроде `Utils2FinalReally.js`

Викторина

Какой признак больше всего похож на big ball of mud?

1. Модули Auth и Orders с понятными именами и малым числом связей
2. Любое мелкое изменение требует правок в десятках несвязанных мест
3. Один деплой всего приложения
4. Наличие базы данных

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Хаотичное влияние изменений — ядро антипаттерна.

(3) — просто монолит; сам по себе не грязь.

Трогаем одно место vs много

Задайте вопрос перед фичей:

1. Какие модули **должны** измениться?
2. Какие ещё «зацепятся» случайно?
3. Можно ли сузить зацепки, проведя границу иначе?

Это мост к maintainability из недели 2.

Сцепление (*coupling*) — простыми словами

Сцепление — насколько модули *зависят* друг от друга.

- Высокое сцепление: правка А ломает В и С
- Более низкое: А говорит с В через узкий понятный «вход»

Не цель — «ноль связей» (тогда не система), а **осознанные** связи.

Связность (*cohesion*) — простыми словами

Связность — насколько вещи *внутри* модуля про одну тему.

- Высокая: в Payments — про оплату, а не про аватарки
- Низкая: файл «делает всё на свете»

Запомните

Хотим: **высокая связность** внутри модуля, **умеренное сцепление** между модулями.

Мини-пример: столовая

Фичи: меню, корзина, оплата, очередь на кухне, уведомление «готово».

- Плохо: один класс `AppManager` на всё
- Лучше: `Menu`, `Orders`, `KitchenQueue`, `Payments`, `Notify`
- Связь: `Orders` спрашивает `Menu`; `Payments` подтверждает `Orders` — не наоборот «в обход»

Викторина

«В модуле Notifications только отправка писем и push, без логики цен» — это скорее про:

1. Высокую связность (*cohesion*)
2. Обязательный big ball of mud
3. Запрет монолита
4. Только про цвет UI

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

1. Высокая cohesion: вещи внутри модуля про одну ответственность.

Викторина

Верно или неверно?

Монолит всегда означает, что любое изменение трогает все файлы проекта.

1. Верно
2. Неверно

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

Неверно. В модульном монолите изменение часто локализуется в одном-двух модулях — если границы живые.

Оставьте простой (даже слабо модульный) монолит, если:

- Команда крошечная, срок короткий
- Домен ещё не ясен (рано резать «не туда»)
- Операции «один деплой» важнее гибкости кусков

Но: даже тогда полезно **назвать** будущие модули на схеме.

Что делать на практике сегодня

1. Взять свалку фич
2. Предложить модули (имя + ответственность)
3. Отметить болевые точки, если границ нет
4. Написать, когда бы оставили простой монолит

1. Монолит — про как **деплоим**, не про «плохой код»
2. Модульный монолит — один деплой + границы внутри
3. Big ball of mud — нет границ, любое изменение лотерея
4. Смотрим влияние изменений; целимся в высокую cohesion и умеренный coupling
5. Иногда простой монолит — правильный trade-off

монолит (*monolith*) · модуль (*module*) · модульный монолит (*modular monolith*)
большой ком грязи (*big ball of mud*)
сцепление (*coupling*) · связность (*cohesion*) · влияние изменений

Практика (сегодня)

Раздатка 03-week03-practice.md: фичи → модули → боли → рефлексия

Следующая лекция (неделя 4)

Клиент-сервер (*client-server*)

Что ещё мутно: монолит, модули или big ball of mud?